



RaqeemOS

نظام المطبعة الأوروبية – التوثيق التقني الكامل

RaqeemOS – Technical Documentation

EDITION 2026

إصدار البرنامج 0.1.0

نظرة عامة على المشروع

نوع المشروع

تطبيق سطح مكتب (Desktop App) مبني على **Electron**، يحتوي بداخله على واجهة ويب حديثة (**React SPA**) و خادم API محلي (**Node.js / Express**) يعملان معًا داخل نفس العملية — بدون أي اعتماد على خادم سحابي خارجي. يشتغل التطبيق محليًا بالكامل على جهاز الزبون، وقاعدة البيانات ملف SQLite واحد على نفس الجهاز.

البنية العامة (Architecture)

الخادم (Server)

Express API محلي يشتغل داخل عملية Electron الرئيسية نفسها، على منفذ محلي (localhost).

الواجهة (Client)

React 18 + Vite — تُبنى كملفات ثابتة (HTML/CSS/JS) وتُحمّل داخل نافذة Electron.

ملاحظة دقة: هذا المستند مبني على فحص مباشر لملفات المشروع الفعلية (package.json، الكود المصدري، الإعدادات) — وليس على افتراضات عامة عن نوع المشاريع المشابهة.

اللغات البرمجية المستخدمة فعلياً

الطبقة	اللغة/الصيغة	ملاحظة
الواجهة (Frontend)	JavaScript (JSX)	مكونات React بصيغة <code>jsx</code> . — لا يوجد TypeScript فعلياً بالمشروع (لا <code>tsconfig.json</code> ولا ملفات <code>.tsx</code>)
الخادم (Backend)	Node.js (CommonJS)	ملفات <code>.js</code> . بصيغة <code>require/module.exports</code>
عملية Electron الرئيسية	Node.js (CommonJS)	Main Process — وصول كامل لنظام التشغيل (ملفات، Hardware ID، إلخ)
التنسيق (Styling)	CSS + Tailwind CSS	Utility classes مع بعض CSS مخصص بملف <code>index.css</code>
البنية (Markup)	HTML5	صفحة واحدة (<code>index.html</code>) — تطبيق Single Page Application
قاعدة البيانات	SQL (SQLite dialect)	ملف <code>schema.sql</code> يعرّف الجداول

تصحيح مهم: لا يستخدم هذا المشروع TypeScript — كل الكود جافاسكربت عادي (JSX). أي توثيق سابق يفترض وجود `App.tsx` أو `tsconfig.json` غير دقيق بخصوص هذا المشروع تحديداً.

التقنيات والأطر الأساسية

التقنية	الإصدار الفعلي	الدور
React	^18.3.1	مكتبة بناء واجهة المستخدم
Vite	^5.4.11	أداة البناء والتطوير (Dev Server + Bundler)
Electron	^33.2.1	تغليف التطبيق كبرنامج سطح مكتب
Tailwind CSS	^3.4.17	تنسيق الواجهة (Utility-first CSS)
React Router DOM	^6.28.1	التنقل بين الصفحات (HashRouter)
Framer Motion	^11.18.2	الحركات والانتقالات البصرية
Express	^4.21.2	خادم الـ API المحلي
better-sqlite3	^12.11.1	محرك قاعدة البيانات المحلية

لا توجد بالمشروع مكتبة أيقونات مخصصة (مثل Lucide) — الأيقونات المستخدمة بالواجهة هي رموز إيموجي مباشرة (👤 📄 🖨️ ...) أو SVG مرسومة يدويًا داخل المكونات.

قائمة كاملة بالمكتبات المستخدمة

مكتبات الواجهة (Client)

المكتبة	الإصدار	الاستخدام	الترخيص
axios	^1.7.9	طلبات HTTP و API	MIT
framer-motion	^11.18.2	حركات وانتقالات	MIT
html5-qrcode	^2.3.8	مسح الباركود بالكاميرا	Apache-2.0
qrcode	^1.5.4	توليد رموز QR	MIT
xlsx (SheetJS)	^0.18.5	تصدير تقارير Excel	Apache-2.0

مكتبات الخادم/ Electron (Server + Main Process)

المكتبة	الإصدار	الاستخدام	الترخيص
bcryptjs	^2.4.3	تشفير كلمات السر وأرقام PIN	MIT
jsonwebtoken	^9.0.2	جلسات تسجيل الدخول (JWT)	MIT
express-rate-limit	^8.7.0	الحد من الطلبات (حماية API)	MIT
multer	^1.4.5-lts.1	رفع الملفات (تصاميم/شعار)	MIT
cors	^2.8.5	سياسات مشاركة الموارد	MIT
electron-store	^8.2.0	تخزين إعدادات/ترخيص مشفر محليًا	MIT
electron-updater	^6.8.9	التحديث التلقائي عبر GitHub Releases	MIT
node-machine-id	^1.1.12	توليد بصمة جهاز فريدة (للترخيص)	Apache-2.0
whatsapp-web.js	^1.34.7	إرسال رسائل واتساب تلقائية	Apache-2.0
electron-builder	^25.1.8	بناء ملف التثبيت (Setup.exe)	MIT

قاعدة البيانات والتخزين المحلي

لا توجد أي خدمة سحابية

هذا المشروع لا يستخدم Supabase ولا Firebase ولا أي قاعدة بيانات سحابية — كل البيانات محفوظة محليًا 100% على جهاز الزبون. هذا قرار تصميمي مقصود (يعمل بدون إنترنت، خصوصية كاملة للبيانات).

النوع	التقنية	الاستخدام
قاعدة البيانات الرئيسية	SQLite (better-sqlite3)	الطلبات، الزبائن، المخزون، الفواتير، الموظفين
إعدادات وترخيص	electron-store (AES encrypted)	مفتاح الترخيص، كلمة سر الحماية الإضافية
ملفات مرفوعة	نظام ملفات محلي	شعار المطبعة، تصاميم الطلبات
النسخ الاحتياطي	مشفّر db. ملف	نسخة يومية تلقائية + نسخ يدوية عند الطلب

مفتاح تشفير electron-store مُشتق من Hardware ID الجهاز نفسه (انظر [electron/secret.js](#)) — فحقى لو انسخ أحد ملف الإعدادات لجهاز آخر، ما يقدر يفكّه بدون نفس الجهاز الأصلي.

نظام الترخيص والحماية

الفكرة العامة

نظام ترخيص محلي بالكامل (بدون سيرفر تحقق خارجي)، مبني على تشفير مفتاح عام/خاص RSA-2048، مربوط ببصمة الجهاز (Hardware ID) الحقيقية — البرنامج مجاني بالكامل، والهدف من النظام هو ربط كل نسخة بجهاز واحد فقط، مو تحصيل دفعات.

كيف يشتغل

- عند أول تشغيل: يُولد البرنامج Hardware ID حقيقي عبر `node-machine-id`.
- فترة تجريبية مجانية: **14 يوم** كاملة بدون أي مفتاح.
- يرسل صاحب المطبعة الـ Hardware ID عبر واتساب لصاحب البرنامج.
- يُولّد مفتاح تفعيل موقع بـ RSA-SHA256 عبر أداة محلية منفصلة (`licensing-vendor/generate-license.js`) — المفتاح الخاص لا يُشحن أبدًا مع البرنامج.
- مدة صلاحية المفتاح الافتراضية: مدى الحياة (٣٦٥٠٠ يوم ≈ ١٠٠ سنة) — سياسة مجانية دائمة.
- يمكن نقل الترخيص لجهاز جديد عبر زر "نقل الترخيص لجهاز جديد" بالإعدادات.

العنصر	التفاصيل
خوارزمية التوقيع	RSA-2048 / SHA-256
بصمة الجهاز	node-machine-id (machineIdSync)
التخزين المحلي	electron-store (مشفّر)
الفترة التجريبية	14 يوم
مدة الترخيص المفعل	مدى الحياة (مجاني)

شجرة مجلدات المشروع

```

RaqeemOS/
├── electron/      Electron main process
│   ├── main.js   IPC handlers + نافذة التطبيق
│   ├── license.js RSA نظام الترخيص
│   ├── preload.js Electron جسر آمن بين الواجهة و
│   ├── backup.js النسخ الاحتياطي المشفر
│   ├── secret.js اشتقاق مفتاح التشفير المحلي
│   └── updater.js التحديث التلقائي
├── server/        محلي Express API
│   ├── index.js
│   ├── db/        schema.sql + db.js
│   ├── routes/     13 ملف (orders, customers, stock...)
│   ├── middleware/ auth.js
│   └── services/    whatsappService.js, settingsService.js
├── client/        React + Vite SPA
│   ├── src/
│   │   ├── pages/  صفحة 20
│   │   ├── components/  مكون 18 (+ui/)
│   │   ├── context/ Auth, Language, Theme, Dashboard
│   │   ├── hooks/   useLicense
│   │   ├── config/   nav.js
│   │   ├── locales/  ar.json, en.json
│   │   └── utils/     format.js, whatsapp.js
│   ├── tailwind.config.js
│   └── vite.config.js
├── licensing-vendor/  أداة توليد المفاتيح (لا تشحن للزبون)
├── scripts/           smoke-test.js
└── package.json       (البناء Electron + scripts) جذر المشروع

```


متطلبات التشغيل والتوزيع

للمستخدم النهائي (النسخة المبنية)

المتطلب	التفاصيل
نظام التشغيل	Windows 10 / 11 (64-bit)
الإنترنت	غير مطلوب للعمل اليومي — مطلوب فقط لإرسال واتساب أو التحديث التلقائي
التثبيت	ملف NSIS Setup.exe (يتيح اختيار مسار التثبيت)
المساحة	حسب حجم قاعدة البيانات والنسخ الاحتياطية المتراكمة محليًا

بيئة التطوير

الأداة	الإصدار المستخدم بالتطوير
Node.js	22.23.2
npm	Node.js مرفق مع
أوامر التشغيل	npm run dev (client+server) · npm run dev:electron
البناء النهائي	npm run dist (electron-builder → Setup.exe)

ملاحظة تقنية حقيقية من الفحص: مكتبة **better-sqlite3** Native Module يجب إعادة بنائها (**npm rebuild** أو **electron-rebuild**) بشكل مختلف حسب تشغيل الكود بـ Node.js العادي أو داخل Electron — النسختان تستخدمان إصدار ABI مختلف.